In [4]:
```
#Q1

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets
from torchvision.transforms import ToTensor
from torch.utils.data import DataLoader

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")

batch_size=64
learning_rate=0.001
epochs=2

class CNNClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv_layer=nn.Sequential(
            nn.Conv2d(1,32,kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32,64,kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc_layer=nn.Sequential(
            nn.Linear(64*5*5,128),
            nn.ReLU(),
            nn.Linear(128,10)
        )
    def forward(self,x):
        x=self.conv_layer(x)
        x=x.view(x.size(0),-1)
        x=self.fc_layer(x)
        return x

train_dataset=datasets.MNIST(root="./data",train=True,transform=ToTensor(

train_loader=DataLoader(train_dataset,batch_size=batch_size,shuffle=True)

model=CNNClassifier().to(device)
loss_fn=nn.CrossEntropyLoss()
optimizer=optim.SGD(model.parameters(),lr=learning_rate)

for epoch in range(epochs):
    print(f"Epoch {epoch+1}/{epochs}")
    for images,labels in train_loader:
        images,labels=images.to(device),labels.to(device)
        outputs=model(images)
        loss=loss_fn(outputs,labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print("Loss: ",loss.item())
```

```
import os
os.makedirs("./ModelFiles",exist_ok=True)

torch.save({
    'model_state_dict':model.state_dict(),
    'optimizer_state_dict':optimizer.state_dict()
},"./ModelFiles/model.pt")

print("Model Saved Successfully")


test_dataset=datasets.FashionMNIST(root='./data',train=False,transform=To
test_loader=DataLoader(test_dataset,batch_size=batch_size,shuffle=False)

model=CNNClassifier()
checkpoint=torch.load("./ModelFiles/model.pt",map_location=device)

model.load_state_dict(checkpoint["model_state_dict"])
model.to(device)

print("Model Loaded Successfully!")

print("Models state dict:")
for param_tensor in model.state_dict():
    print(param_tensor,"\t",model.state_dict()[param_tensor].size())
print()

model.eval()
correct=0
total=0

with torch.no_grad():
    for images,labels in test_loader:
        images, labels=images.to(device),labels.to(device)
        outputs=model(images)
        _,predicted=torch.max(outputs,1)
        total+=labels.size(0)
        correct+=(predicted==labels).sum().item()

accuracy=100.0*correct/total
print(f"Overall accuracy on FashionMnist is {accuracy:.2f}%")
```

```
Epoch 1/2
Loss:  2.269801139831543
Epoch 2/2
Loss:  2.218413829803467
Model Saved Successfully
Model Loaded Successfully!
Models state dict:
conv_layer.0.weight      torch.Size([32, 1, 3, 3])
conv_layer.0.bias        torch.Size([32])
conv_layer.3.weight      torch.Size([64, 32, 3, 3])
conv_layer.3.bias        torch.Size([64])
fc_layer.0.weight        torch.Size([128, 1600])
fc_layer.0.bias          torch.Size([128])
fc_layer.2.weight        torch.Size([10, 128])
fc_layer.2.bias          torch.Size([10])

Overall accuracy on FashionMnist is 14.94%
```

In [8]:
```
#Q2
import torch
from torch.utils.data import DataLoader
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms, datasets
from torchvision.models import alexnet, AlexNet_Weights


weights = AlexNet_Weights.DEFAULT
model = alexnet(weights=weights)

for param in model.features.parameters():
    param.requires_grad = False

model.classifier[6] = nn.Linear(4096, 2)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)


train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomCrop(227),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(227),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])


data_dir = "cats_and_dogs_filtered"

train_dataset = datasets.ImageFolder(
    root=data_dir + "/train",
    transform=train_transform
)

val_dataset = datasets.ImageFolder(
    root=data_dir + "/validation",
    transform=val_transform
)

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=4, shuffle=False)


loss_fn = nn.CrossEntropyLoss()

optimizer = optim.SGD(model.classifier.parameters(),
                      lr=0.001, momentum=0.9)
```

```
epochs = 5

for epoch in range(epochs):
    model.train()
    running_loss = 0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    train_acc = 100 * correct / total

    model.eval()
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    val_acc = 100 * val_correct / val_total

    print(f"Epoch [{epoch+1}/{epochs}] "
          f"Loss: {running_loss/len(train_loader):.4f} "
          f"Train Acc: {train_acc:.2f}% "
          f"Val Acc: {val_acc:.2f}%")

torch.save(model.state_dict(), "alexnet_cats_dogs.pth")
print("Training Complete. Model Saved Successfully")
```

```
Epoch [1/5] Loss: 0.4662 Train Acc: 90.25% Val Acc: 95.10%
Epoch [2/5] Loss: 0.2314 Train Acc: 93.65% Val Acc: 94.90%
Epoch [3/5] Loss: 0.1266 Train Acc: 95.60% Val Acc: 96.10%
Epoch [4/5] Loss: 0.0948 Train Acc: 96.65% Val Acc: 95.00%
Epoch [5/5] Loss: 0.0652 Train Acc: 97.70% Val Acc: 96.00%
Training Complete. Model Saved Successfully
```

In [10]:

```
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from torchvision import datasets
from torchvision.transforms import ToTensor
from torch.utils.data import DataLoader
import os

#Q1 code
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

batch_size = 64
learning_rate = 0.001
INITIAL_EPOCHS = 2
TOTAL_EPOCHS = 5

class CNNClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv_layer = nn.Sequential(
            nn.Conv2d(1,32,3),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32,64,3),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc_layer = nn.Sequential(
            nn.Linear(64*5*5,128),
            nn.ReLU(),
            nn.Linear(128,10)
        )

    def forward(self,x):
        x = self.conv_layer(x)
        x = x.view(x.size(0),-1)
        x = self.fc_layer(x)
        return x

train_dataset = datasets.MNIST(root="./data",train=True,
                               transform=ToTensor(),download=True)

train_loader = DataLoader(train_dataset,
                          batch_size=batch_size,shuffle=True)

model = CNNClassifier().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(),lr=learning_rate)


os.makedirs("./checkpoints",exist_ok=True)

best_loss = float('inf')

for epoch in range(INITIAL_EPOCHS):
    model.train()
    running_loss = 0

    for images,labels in train_loader:
        images,labels = images.to(device),labels.to(device)

        outputs = model(images)
```

```
        loss = loss_fn(outputs,labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{INITIAL_EPOCHS}] Loss: {avg_loss:.4f}")

    # Save best model
    if avg_loss < best_loss:
        best_loss = avg_loss
        torch.save(model.state_dict(),
                   "./checkpoints/best_model.pt")
        print("Best model saved!")

checkpoint = {
    "last_loss": avg_loss,
    "last_epoch": INITIAL_EPOCHS,
    "model_state": model.state_dict(),
    "optimizer_state": optimizer.state_dict()
}

torch.save(checkpoint,"./checkpoints/checkpoint.pt")
print("Checkpoint saved!")


print("\nResuming from checkpoint\n")

checkpoint = torch.load("./checkpoints/checkpoint.pt")

model = CNNClassifier().to(device)
model.load_state_dict(checkpoint["model_state"])

optimizer = optim.SGD(model.parameters(),lr=learning_rate)
optimizer.load_state_dict(checkpoint["optimizer_state"])

start_epoch = checkpoint["last_epoch"]
best_loss = checkpoint["last_loss"]

for epoch in range(start_epoch, TOTAL_EPOCHS):
    model.train()
    running_loss = 0

    for images,labels in train_loader:
        images,labels = images.to(device),labels.to(device)

        outputs = model(images)
        loss = loss_fn(outputs,labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{TOTAL_EPOCHS}] Loss: {avg_loss:.4f}")
```

```
    if avg_loss < best_loss:
        best_loss = avg_loss
        torch.save(model.state_dict(),
                   "./checkpoints/best_model.pt")
        print("New Best model saved")

print("\nTraining Completed")
```

```
Epoch [1/2] Loss: 2.2695
Best model saved!
Epoch [2/2] Loss: 2.1495
Best model saved!
Checkpoint saved!

Resuming from checkpoint

Epoch [3/5] Loss: 1.6679
New Best model saved
Epoch [4/5] Loss: 0.8932
New Best model saved
Epoch [5/5] Loss: 0.5817
New Best model saved

Training Completed
```